Cetus

Audit Report

contact@bitslab.xyz

https://twitter.com/movebit_

Mon Jun 16 2025

Cetus Audit Report

1 Executive Summary

1.1 Project Information

Description	Cetus is a pioneer DEX and concentrated liquidity protocol built on the Sui and Aptos blockchain.
Type	DEX
Auditors	MoveBit
Timeline	Thu May 29 2025 - Fri May 30 2025
Languages	Move
Platform	Sui
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/CetusProtocol/cetus-clmm-sui
Commits	11ea0aec59a8edcf813a744797e64f58cf84d99c 8437040e7a22bf11c8525d801d83f37300dc7d90

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
FAC	cetus_clmm/sources/factory.move	2c33a0e736d676c932f61f5bc81dc7a011a474a5
CON1	cetus_clmm/sources/config.move	4457fe4aab571cfd1473830a1ab629332bc9b6b0
PCR1	cetus_clmm/sources/pool_creator.move	a75f3e9f75b998b26d35ad0e5073b433d9e76011
FAC	cetus_clmm/sources/factory.move	8776e4ba8751d2072fd8d9272c66ebae197583af
POO1	cetus_clmm/sources/pool.move	84aaa2c463de58c497629ffefeb3de539f1bb065
PSN	cetus_clmm/sources/positon_snapshot.move	f023af7c169ddf8e5dbeaf29aa1740f27acc4355

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	3	2	1
Informational	1	0	1
Minor	1	1	0
Medium	1	1	0
Major	0	0	0
Critical	0	0	0

1.4 MoveBit Audit Breakdown

MoveBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values
- The flow of capability
- Witness Type

1.5 Methodology

The security team adopted the **"Testing and Automated Analysis"**, **"Code Review"** and **"Formal Verification"** strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Formal Verification(Optional)

Perform formal verification for key functions with the Move Prover.

(4) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by Cetus to identify any potential issues and vulnerabilities in the source code of the Cetus smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 3 issues of varying severity, listed below.

ID	Title	Severity	Status
FAC-1	Code Optimization	Minor	Fixed
FAC-2	Incorrect Error Code	Informational	Acknowledged
PSN-1	Incorrect Rounding Down in `calculate_remove_liquidity()`	Medium	Fixed

3 Participant Process

Here are the relevant actors with their respective abilities within the Cetus Smart Contract :

Owner

- The owner can call the `add_allowed_list` / `add_denied_list` function to add a coin into `allowed_list` or `denied_list` .
- The owner can call the `mint_pool_creation_cap_by_admin` to get PoolCreationCap directly.

User

- Users can call the `mint_pool_creation_cap` function to get `PoolCreationCap` with their `TreasuryCap` .
- Users can call the `register_permission_pair` and `unregister_permission_pair` functions to manage the pool.

4 Findings

FAC-1 Code Optimization

Severity: Minor

Status: Fixed

Code Location:

cetus_clmm/sources/factory.move#849,850

Descriptions:

In the conditional logic, the `=` check is redundant. You can first determine whether the values are equal, and then simply check if `len_a > len_b`. The same applies to cases like 886 and 887.

Suggestion:

It is recommended to check whether the values are equal, and then if `len_a > len_b`.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

FAC-2 Incorrect Error Code

Severity: Informational

Status: Acknowledged

Code Location:

cetus_clmm/sources/factory.move#752

Descriptions:

In the `factory.move` #752, the error code `EPoolAlreadyExist` is not correct.

Suggestion:

It is recommended to update the error code.

PSN-1 Incorrect Rounding Down in `calculate_remove_liquidity()`

Severity: Medium

Status: Fixed

Code Location:

cetus_clmm/sources/positon_snapshot.move#79

Descriptions:

The method `calculate_remove_liquidity()` currently uses floor rounding when calculating the amount of liquidity to remove, which is incorrect. To prevent protocol losses, it should use ceiling rounding instead.

Suggestion:

It is recommended to use ceiling rounding instead.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.

